

SISTEMAS OPERATIVOS: Diseño e Implementación

SEGUNDA EDICION

ANDREW S. TANENBAUM

Vrije Universiteit
Amsterdam, Países Bajos

ALBERT S. WOODHULL

Hampshire College
Amherst, Massachusetts

INTRODUCCION

Sin su software, la computadora es básicamente un montón de metal inútil. Con su software, una computadora puede almacenar, procesar y recuperar información; exhibir documentos multimedia; realizar búsquedas en Internet; y realizar muchas otras actividades valiosas para justificar su existencia. El software de computadora puede dividirse a grandes rasgos en dos tipos: programas de sistema, que controlan la operación de la computadora misma, y programas de aplicación, que realizan las tareas reales que el usuario desea. El programa de sistema más fundamental es el **sistema operativo**, que controla todos los recursos de la computadora y establece la base sobre la que pueden escribirse los programas de aplicación.

Un sistema de computadora moderno consiste en uno o más procesadores, memoria principal (también conocida como RAM, memoria de acceso aleatorio), discos, impresoras, interfaces de red y otros dispositivos de entrada/salida. A todas luces, se trata de un sistema complejo. Escribir programas que sigan la pista a todos estos componentes y los usen correctamente, ya no digamos óptimamente, es una tarea en extremo difícil. Si todos los programadores tuvieran que ocuparse de cómo trabajan las unidades de disco, y de las docenas de cosas que pueden fallar al leer un bloque de disco, es poco probable que pudieran escribirse muchos programas.

Hace muchos años se hizo muy evidente que debía encontrarse alguna forma de proteger a los programadores de la complejidad del hardware. La solución que ha evolucionado gradualmente consiste en poner una capa de software encima del hardware solo, que se encargue de administrar todas las partes del sistema y presente al usuario una interfaz o **máquina virtual** que sea más fácil de entender y programar. Esta capa de software es el sistema operativo, y constituye el tema de este libro.

La situación se muestra en la Fig. 1-1. En la parte inferior está el hardware que, en muchos casos, también se compone de dos o más capas. La capa más baja contiene los dispositivos físicos, que consisten en chips de circuitos integrados, alambres, fuentes de potencia, tubos de rayos catódicos y otros aparatos físicos similares. La forma en que éstos se construyen y sus principios de funcionamiento pertenecen al campo del ingeniero electricista.

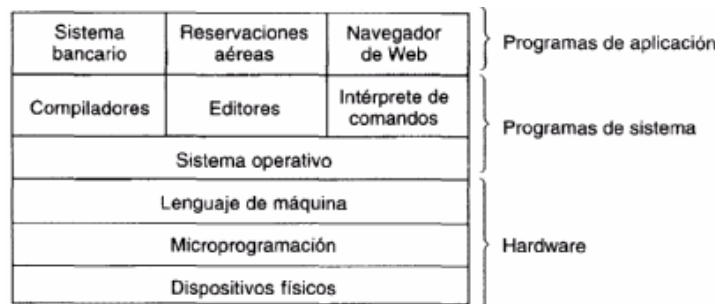


Figura 1-1. Un sistema de computadora consiste en hardware, programas de sistema y programas de aplicación.

A continuación (en algunas máquinas) viene una capa de software primitivo que controla directamente estos dispositivos y ofrece una interfaz más aseada a la siguiente capa. Este software, llamado **microprograma**, suele estar almacenado en memoria de sólo lectura. En realidad es un intérprete, que obtiene las instrucciones de lenguaje de máquina como ADD, MOVE y JUMP y las ejecuta en una serie de pasos pequeños. Por ejemplo, para ejecutar una instrucción ADD (sumar), el microprograma debe determinar dónde se encuentran los números que se van a sumar, obtenerlos, sumarlos y almacenar el resultado en algún lugar. El conjunto de instrucciones que el microprograma interpreta define el **lenguaje de máquina**, que no es realmente parte de la máquina física, aunque los fabricantes de computadoras siempre lo describen en sus manuales como tal, de modo que muchas personas piensan en él como si fuera la "máquina" real.

Algunas computadoras, llamadas **RISC (computadoras con conjunto de instrucciones reducido)**, no tienen un nivel de microprogramación. En estas máquinas, el hardware ejecuta las instrucciones de lenguaje de máquina directamente. Por ejemplo, el Motorola 680x0 tiene un nivel de microprogramación, pero el IBM PowerPC no.

El lenguaje de máquina por lo regular cuenta con entre 50 y 300 instrucciones, la mayor parte de ellas para

trasladar datos dentro de la máquina, realizar operaciones aritméticas y comparar valores. En esta capa, los dispositivos de entrada/salida se controlan cargando valores **registros de dispositivo** especiales. Por ejemplo, para un disco que ejecuta una lectura, sus registros se cargan con los valores de dirección del disco, dirección de memoria principal, conteo de bytes y dirección de acceso (READ o WRITE). En la práctica se requieren muchos parámetros más, y el estado devuelto por la unidad de disco después de una operación es muy complejo. Además, la temporización desempeña un papel importante en la programación de muchos dispositivos de E/S.

Una función importante del sistema operativo es ocultar toda esta complejidad y ofrecer al programador un conjunto de instrucciones más cómodo con el que pueda trabajar. Por ejemplo, LEER BLOQUE DE ARCHIVO es conceptualmente más sencillo que tener que preocuparse por los detalles de mover cabezas de disco, esperar que se estabilicen, etcétera.

Encima del sistema operativo está el resto del software de sistema. Aquí encontramos el intérprete de comandos (**shell**), sistemas de ventanas, compiladores, editores y otros programas similares independientes de la aplicación. Es importante darse cuenta de que estos programas definitivamente no forman parte del sistema operativo, a pesar de que casi siempre son provistos por el fabricante de la computadora. Éste es un punto crucial, aunque sutil. El sistema operativo es la porción del software que se ejecuta en **modo kernel** o **modo supervisor**, y está protegido por el hardware contra la intervención del usuario (olvidándonos por el momento de algunos de los microprocesadores más viejos que no tienen ninguna protección de hardware). Los compiladores y editores se ejecutan en **modo de usuario**. Si a un usuario no le gusta un compilador en particular, él¹ está en libertad de escribir el suyo propio si lo desea; no está en libertad de escribir su propio manejador de interrupciones del disco, que forma parte del sistema operativo y normalmente está protegido por el hardware contra los intentos de los usuarios por modificarlo.

Por último, encima de los programas de sistema vienen los programas de aplicación. Los usuarios compran o escriben estos programas para resolver sus problemas particulares, como procesamiento de textos, hojas de cálculo, cálculos de ingeniería o juegos.

1.1 ¿QUÉ ES UN SISTEMA OPERATIVO?

La mayoría de los usuarios de computadora han tenido algo de experiencia con un sistema operativo, pero no es fácil precisar con exactitud qué es un sistema operativo. Parte del problema consiste en que el sistema operativo realiza dos funciones que básicamente no están relacionadas entre sí y, dependiendo de a quién le preguntemos, por lo general se nos habla principalmente de una función o de la otra. Veamos ahora las dos.

1.1.1 El sistema operativo como máquina extendida

Como ya dijimos, la **arquitectura** (conjunto de instrucciones, organización de memoria, E/S y estructura de buses) de la mayor parte de las computadoras en el nivel de lenguaje de máquina es primitiva y difícil de programar, sobre todo para entrada/salida. A fin de hacer más concreto este punto, veamos brevemente cómo se realiza la E/S de disco flexible usando el chip controlador NEC PD765 (o su equivalente), utilizado por la mayor parte de las computadoras personales. (En todo este libro usaremos indistintamente los términos “*disco flexible*” y “*disquete*”.) El PD765 tiene 16 comandos, cada uno de los cuales se especifica cargando entre 1 y 9 bytes en un registro de dispositivo. Estos comandos sirven para leer y escribir datos, mover el brazo del disco y formatear pistas, así como para inicializar, detectar, restablecer y recalibrar el controlador y las unidades de disco.

Los comandos más básicos son READ y WRITE, cada uno de los cuales requiere 13 parámetros empacados en 9 bytes. Estos parámetros especifican cosas tales como la dirección del bloque de disco que se va a leer, el número de sectores por pista, el modo de grabación empleado en el medio físico, el espaciado de la brecha entre sectores y qué hacer con una marca de “dirección de datos eliminada”. Si usted no entiende a qué nos referimos, no se preocupe; de eso se trata precisamente: es algo muy esotérico. Cuando se completa la operación, el chip controlador devuelve 23 campos de estado y error empacados en 7 bytes. Por si esto no fuera suficiente, el programador del disco flexible también debe tener presente en todo momento si el motor está encendido o apagado. Si el motor está apagado, debe encenderse (con un retardo de arranque largo) antes de que puedan leerse o escribirse datos. Empero, el motor no puede dejarse encendido demasiado tiempo, pues el disco flexible se desgastaría. Por tanto, el programador debe encontrar un equilibrio entre los retardos de arranque largos y el desgaste de los discos flexibles (y la pérdida de los datos que contienen).

Sin entrar en los verdaderos detalles, debe quedar claro que el programador ordinario seguramente no

1 “Él” debe leerse “él o ella” a lo largo de todo el libro

quiere intervenir de manera demasiado íntima en la programación de los discos flexibles (o de los duros, que son igualmente complejos, y muy distintos). En vez de ello, lo que el programador quiere es manejar una abstracción sencilla, de alto nivel. En el caso de los discos, una abstracción típica sería que el disco contiene una colección de archivos con nombre. Cada archivo puede abrirse para lectura o escritura, leerse o escribirse, y por último cerrarse. Los detalles de si la grabación debe usar o no modulación de frecuencia modificada y cuál es la situación actual del motor no deberán aparecer en la abstracción presentada al usuario.

El programa que oculta la verdad acerca del hardware y presenta al programador una vista sencilla y bonita de archivos con nombre que pueden leerse y escribirse es, por supuesto, el sistema operativo. Así como el sistema operativo aísla al programador del hardware del disco y presenta una interfaz sencilla orientada a archivos, también oculta muchos asuntos desagradables referentes a interrupciones, temporizadores, administración de memoria y otras funciones de bajo nivel. En cada caso, la abstracción que el sistema operativo ofrece es más sencilla y fácil de usar que el hardware subyacente.

En esta vista, la función del sistema operativo es presentar al usuario el equivalente de una **máquina extendida** o **máquina virtual** que es más fácil de programar que el hardware subyacente. La forma en que el sistema operativo logra este objetivo es una historia larga, que estudiaremos con detalle a lo largo del libro.

1.1.2 El sistema operativo como administrador de recursos

El concepto del sistema operativo como algo cuya función primordial es ofrecer a los usuarios una Interfaz cómoda es una visión descendente. Una visión ascendente alternativa postula que el sistema operativo está ahí para administrar todos los componentes de un sistema complejo. Las computadoras modernas constan de procesadores, memorias, temporizadores, discos, ratones, interfaces con redes, impresoras láser y una gran variedad de otros dispositivos. En la visión alternativa, la misión del sistema operativo es asegurar un reparto ordenado y controlado de los procesadores, memorias y dispositivos de E/S entre los diferentes programas que compiten por ellos.

Imagine lo que sucedería si tres programas que se ejecutan en alguna computadora trataran de imprimir sus salidas simultáneamente en la misma impresora. Las primeras líneas del listado podrían ser del programa 1, las siguientes del programa 2, luego algunas del programa 3, y así sucesivamente. El resultado sería un caos. El sistema operativo puede poner orden en el caos potencial almacenando temporalmente en el disco todas las salidas destinadas para la impresora. Cuando un programa haya terminado, el sistema operativo podrá copiar su salida del archivo de disco donde se almacenó a la impresora, mientras que el otro programa puede continuar generando salidas, ajeno al hecho de que dichas salidas no están yendo directamente a la impresora (todavía).

Cuando una computadora (o red) tiene múltiples usuarios, la necesidad de administrar y proteger la memoria, los dispositivos de E/S y demás recursos es aún mayor, ya que de otra manera los usuarios podrían interferirse. Además, es frecuente que los usuarios tengan que compartir no sólo hardware, sino también información (archivos, bases de datos, etc.). En pocas palabras, esta es visión del sistema operativo sostiene que su tarea primordial es seguir la pista de quién está usando cuál recurso, atender solicitudes de recursos, contabilizar la utilización y mediar entre solicitudes en conflicto provenientes de diferentes programas y usuarios.

1.2 HISTORIA DE LOS SISTEMAS OPERATIVOS

Los sistemas operativos han estado evolucionando durante muchos años. En las siguientes secciones examinaremos brevemente este desarrollo. Dado que, históricamente, los sistemas operativos han estado de manera muy estrecha vinculados con la arquitectura de las computadoras en las que se ejecutan, estudiaremos las sucesivas generaciones de computadoras para ver qué clase de sistemas operativos usaban. Esta correspondencia entre las generaciones de sistemas operativos y de computadoras es algo burda, pero establece un poco de estructura que de otra forma sería inexistente.

La primera computadora digital verdadera fue diseñada por el matemático inglés Charles Babbage (1792-1871). Aunque Babbage invirtió la mayor parte de su vida y su fortuna tratando de construir su “máquina analítica”, nunca logró que funcionara correctamente porque era totalmente mecánica, y la tecnología de su época no podía producir las ruedas, engranes y levas con la elevada precisión que él requería. Huelga decir que la máquina analítica no contaba con un sistema operativo.

Como acotación histórica interesante, diremos que Babbage se dio cuenta de que necesitaría software para su máquina analítica, así que contrató a una joven mujer, Ada Lovelace, hija del famoso poeta británico, Lord Byron, como la primera programadora de la historia. El lenguaje de programación Ada® recibió su

nombre en honor a ella.

1.2.1 La primera generación (1945-55): Tubos de vacío y tableros de conmutación

Después del fracaso de los trabajos de Babbage, fueron pocos los avances que se lograron en la construcción de computadoras digitales hasta la Segunda Guerra Mundial. A mediados de la década de 1940, Howard Aiken en Harvard, John von Neumann en el Institute for Advanced Study en Princeton, J. Presper Eckert y William Mauchley en la University of Pennsylvania y Konrad Zuse en Alemania, entre otros, lograron construir máquinas calculadoras usando tubos de vacío. Estas máquinas eran enormes, y ocupaban cuartos enteros con decenas de miles de tubos de vacío, pero eran mucho más lentas que incluso las computadoras personales más baratas de la actualidad.

En esos primeros días, un solo grupo de personas diseñaba, construía, programaba, operaba y mantenía a cada máquina. Toda la programación se realizaba en lenguaje de máquina absoluto, a menudo alambrando tableros de conmutación para controlar las funciones básicas de la máquina. No existían los lenguajes de programación (ni siquiera los de ensamblador). Nadie había oído hablar de los sistemas operativos. La forma de operación usual consistía en que el programador se anotaba para recibir un bloque de tiempo en la hoja de reservaciones colgada en la pared, luego bajaba al cuarto de la máquina, insertaba su tablero de conmutación en la computadora, y pasaba las siguientes horas con la esperanza de que ninguno de los cerca de 20000 tubos de vacío se quemara durante la sesión. Prácticamente todos los problemas eran cálculos numéricos directos, como la producción de tablas de senos y cosenos.

A principios de la década de 1950, la rutina había mejorado un poco con la introducción de las tarjetas perforadas. Ahora era posible escribir programas en tarjetas e introducirlas para ser leídas, en lugar de usar tableros de conmutación; por lo demás, el procedimiento era el mismo.

1.2.2 La segunda generación (1955-65): Transistores y sistemas por lote

La introducción del transistor a mediados de la década de 1950 alteró el panorama radicalmente. Las computadoras se hicieron lo bastante confiables como para poderse fabricar y vender a clientes comerciales con la expectativa de que seguirían funcionando el tiempo suficiente para realizar algo de trabajo útil. Por primera vez, había una separación clara entre diseñadores, constructores, operadores, programadores y personal de mantenimiento.

Estas máquinas se encerraban en cuartos de computadora con acondicionamiento de aire especial, con equipos de operadores profesionales para operarlas. Sólo las grandes empresas, o las principales dependencias del gobierno o universidades, podían solventar el costo de muchos millones de dólares. Para ejecutar un **trabajo** (es decir, un programa o serie de programas), un programador escribía primero el programa en papel (en FORTRAN o ensamblador) y luego lo perforaba en tarjetas. Después, llevaba el grupo de tarjetas al cuarto de entrada y lo entregaba a uno de los operadores.

Cuando la computadora terminaba el trabajo que estaba ejecutando en ese momento, un operador acudía a la impresora, separaba la salida impresa y la llevaba al cuarto de salida donde el programador podía recogerla después. Luego, el operador tomaba uno de los grupos de tarjetas traídos del cuarto de entrada y lo introducía en el lector. Si se requería el compilador de FORTRAN, el operador tenía que traerlo de un archivero e introducirlo en el lector. Gran parte del tiempo de computadora se desperdiciaba mientras los operadores iban de un lugar a otro, en el cuarto de la máquina.

Dado el alto costo del equipo, no es sorprendente que la gente pronto buscara formas de reducir el desperdicio de tiempo. La solución que se adoptó generalmente fue el **sistema por lotes**. El principio de este modo de operación consistía en juntar una serie de trabajos en el cuarto de entrada, leerlos y grabarlos en una cinta magnética usando una computadora pequeña y (relativamente) económica, como una IBM 1401, que era muy buena para leer tarjetas, copiar cintas e imprimir salidas, pero no para realizar cálculos numéricos. Otras máquinas, mucho más costosas, como la IBM 7094, se usaban para la computación propiamente dicha. Esta situación se muestra en la Fig. 1-2.

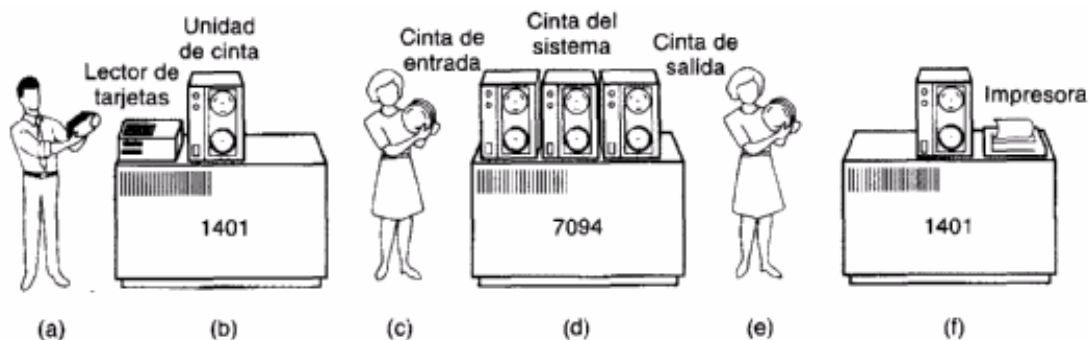


Figura 1-2. Uno de los primeros sistemas por lotes. (a) Los programadores traen tarjetas a la 1401. (b) La 1401 lee lotes de trabajos y los graba en cinta. (c) El operador lleva la cinta de entrada a la 7094. (d) La 7094 realiza la computación. (e) El operador lleva la cinta de salida a la 1401. (f) La 1401 imprime la salida.

Después de cerca de una hora de reunir un lote de trabajos, la cinta se rebobinaba y se llevaba al cuarto de la máquina, donde se montaba en una unidad de cinta. El operador cargaba entonces un programa especial (el antepasado del sistema operativo actual), que leía el primer trabajo de la cinta y lo ejecutaba. La salida se escribía en una segunda cinta, en lugar de imprimirse. Cada vez que terminaba un trabajo, el sistema operativo leía automáticamente el siguiente trabajo de la cinta y comenzaba a ejecutarlo. Una vez que estaba listo todo el lote, el operador dismantaba las cintas de entrada y salida, montaba la cinta de entrada del siguiente lote, y llevaba la cinta de salida a una 1401 para la impresión **fuera de línea** (o sea, no conectada a la computadora principal).

La estructura de un trabajo de entrada típico se muestra en la Fig. 1-3. El trabajo comenzaba con una tarjeta \$JOB, que especificaba el tiempo de ejecución máximo en minutos, el número de cuenta al que se debía cobrar el trabajo, y el nombre del programador. Luego venía una tarjeta \$FORTRAN, que ordenaba al sistema operativo leer el compilador de FORTRAN de la cinta de sistema. Esta tarjeta iba seguida del programa por compilar y por una tarjeta \$LOAD, que ordenaba al sistema operativo cargar el programa objeto recién compilado. (Los programas compilados a menudo se escribían en cintas temporales y tenían que cargarse explícitamente.) Luego venía la tarjeta \$RUN, que ordenaba al sistema operativo ejecutar el programa con los datos que le seguían. Por último, la tarjeta \$END marcaba el final del trabajo. Estas tarjetas de control primitivas eran los precursores de los lenguajes de control de trabajos e intérpretes de comandos modernos.

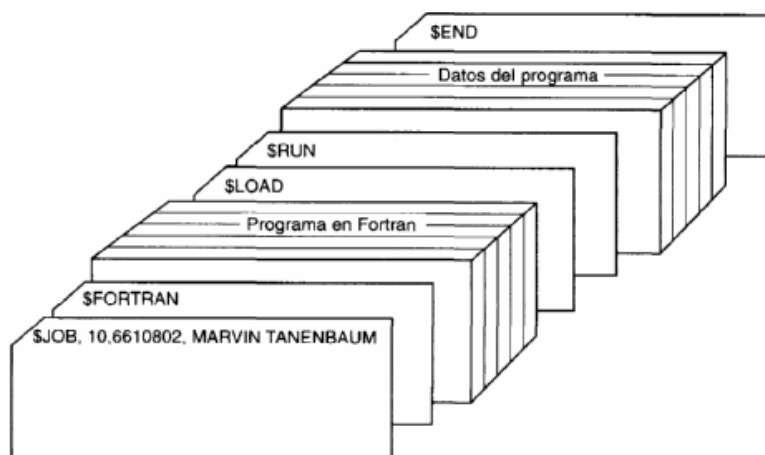


Figura 1-3. Estructura de un trabajo FMS típico

Las computadoras grandes de la segunda generación se usaban primordialmente para cálculos científicos y de ingeniería, como la resolución de ecuaciones diferenciales parciales. Estas máquinas generalmente se programaban en FORTRAN y lenguaje ensamblador. Los sistemas operativos típicos eran FMS (el *Fortran Monitor System*) e IBSYS, el sistema operativo de IBM para la 7094.

1.2.3 La tercera generación (1965-1980): Circuitos integrados y multiprogramación

A principios de la década de 1960, la mayoría de los fabricantes de computadoras tenían dos líneas de producto distintas y totalmente incompatibles. Por un lado estaban las computadoras científicas a gran escala, orientadas hacia las palabras, como la 7094, que se usaban para cálculos numéricos en ciencias e

ingeniería. Por el otro, estaban las computadoras comerciales orientadas hacia los caracteres, como la 1401, que los bancos y las compañías de seguros utilizaban ampliamente para ordenar e imprimir desde cinta.

La creación y mantenimiento de dos líneas de producto totalmente distintas era una situación costosa para los fabricantes. Además, muchos clientes de computadoras nuevas necesitaban inicialmente una máquina pequeña que más adelante les resultaba insuficiente, de modo que querían una máquina más grande que ejecutara todos sus viejos programas, pero más rápidamente.

IBM trató de resolver simultáneamente ambos problemas introduciendo la System/360. La 360 era una serie de máquinas de software compatible que iban desde tamaños comparables a la 1401 hasta computadoras mucho más potentes que la 7094. Las máquinas diferían sólo en el precio y el rendimiento (memoria máxima, velocidad del procesador, número de dispositivos de E/S permitidos, etc.). Puesto que todas las máquinas tenían la misma arquitectura y conjunto de instrucciones, los programas escritos para una máquina podían ejecutarse en todas las demás, al menos en teoría. Además, la 360 estaba diseñada para manejar computación tanto científica como comercial. Así, una sola familia de máquinas podía satisfacer las necesidades de todos los clientes. En años subsecuentes IBM produjo sucesoras comparables a la línea 360, usando tecnología más moderna, conocidas como series 370, 4300, 3080 y 3090.

La 360 fue la primera línea importante de computadoras en usar (a pequeña escala) circuitos integrados (IC), ofreciendo así una ventaja de precio/rendimiento considerable respecto a las máquinas de la segunda generación, que se armaban con transistores individuales. Esta línea fue un éxito inmediato, y la idea de una familia de computadoras compatibles pronto fue adoptada por todos los demás fabricantes importantes. Los descendientes de estas máquinas todavía se emplean en uno que otro centro de cómputo en la actualidad, pero su uso está en rápido declive.

La gran ventaja de la idea de “una familia” fue también su gran debilidad. La intención era que todo el software, incluido el sistema operativo, funcionara en todos los modelos. El software tenía que funcionar en sistemas pequeños, que en muchos casos simplemente sustituían a la 1401 para copiar tarjetas en cinta, y en sistemas muy grandes, que con frecuencia sustituían a las 7094 para realizar pronósticos del tiempo y otros trabajos de computación pesada. El software tenía que ser bueno en sistemas con pocos y con muchos periféricos; tenía que funcionar en entornos comerciales y científicos y, sobre todo, tenía que ser eficiente para todos estos usos distintos.

Era imposible que IBM (o alguien más) pudiera escribir un programa que satisficiera todos esos requisitos opuestos. El resultado fue un sistema operativo enorme, extraordinariamente complejo, tal vez dos o tres órdenes de magnitud mayor que FMS. Este sistema consistía en millones de líneas de lenguaje ensamblador escrito por miles de programadores, y contenía miles y miles de errores, requiriéndose un flujo continuo de nuevas versiones en un intento por corregirlos. Cada versión nueva corregía algunos errores e introducía otros nuevos, de modo que es probable que el número de errores se mantuviera constante con el tiempo.

Uno de los diseñadores de OS/360, Fred Brooks, escribió después un ingenioso e incisivo libro (Brooks, 1975) describiendo sus experiencias con el OS/360. Aunque sería imposible resumir aquí ese libro, baste con decir que la portada muestra una manada de bestias prehistóricas atascadas en un foso de brea. La portada del libro de Silberschatz y Galvin (1994) es una alusión similar.

A pesar de su enorme tamaño y de sus problemas, OS/360 y los sistemas operativos de tercera generación parecidos a él producidos por otros fabricantes de computadoras lograron satisfacer a sus clientes en un grado razonable, y también popularizaron varias técnicas clave que no existían en los sistemas operativos de la segunda generación. Tal vez la más importante de ellas haya sido la **multiprogramación**. En la 7094, cuando el trabajo actual hacía una pausa para esperar que se completara una operación de cinta u otra operación de E/S, la CPU simplemente permanecía ociosa hasta que la E/S terminaba. En los cálculos científicos, con gran uso de CPU, la E/S es poco frecuente, así que el tiempo desperdiciado no es significativo. En el procesamiento de datos comerciales, el tiempo de espera por E/S puede ser el 80 o 90% del tiempo total, de modo que algo debía hacerse para evitar que la CPU estuviera ociosa tanto tiempo.

La solución a la que se llegó fue dividir la memoria en varias secciones, con un trabajo distinto en cada partición, como se muestra en la Fig. 1-4. Mientras un trabajo estaba esperando que terminara su E/S, otro podía estar usando la CPU. Si se podían tener en la memoria principal suficientes trabajos a la vez, la CPU podía mantenerse ocupada casi todo el tiempo. Tener múltiples trabajos en la memoria a la vez requiere hardware especial para proteger cada trabajo contra espionaje por parte de los demás, pero la 360 y otros sistemas de tercera generación estaban equipados con este hardware.



Figura 1-4. Sistema de multiprogramación con tres trabajos en la memoria

Otra característica importante presente en los sistemas operativos de la tercera generación era la capacidad de leer trabajos de las tarjetas al disco tan pronto como se llevaban al cuarto de computadoras. Luego, cada vez que un trabajo terminaba su ejecución, el sistema operativo podía cargar uno nuevo del disco en la partición que había quedado vacía y ejecutarlo. Esta técnica se llama **spooling** (de “*operación simultánea de periféricos en línea*”) y también se usaba para la salida. Con spooling, las 1401 ya no eran necesarias, y desapareció una buena parte del transporte de cintas.

Aunque los sistemas operativos de la tercera generación se adaptaban bien a cálculos científicos extensos y sesiones masivas de procesamiento de datos comerciales, seguían siendo básicamente sistemas por lotes. Muchos programadores añoraban los días de la primera generación cuando tenían toda la máquina para ellos solos durante unas cuantas horas, lo que les permitía depurar sus programas rápidamente. Con los sistemas de la tercera generación, el tiempo entre la presentación de un trabajo y la obtención de las salidas a menudo era de varias horas, y una sola coma mal colocada podía causar el fracaso de una compilación y que el programador desperdiciara medio día.

Este deseo de respuesta rápida preparó el camino para el **tiempo compartido**, una variante de la multiprogramación, en la que cada usuario tiene una terminal en línea. En un sistema de tiempo compartido, si 20 usuarios ingresan en el sistema y 17 de ellos están pensando, hablando o tomando café, la CPU puede asignarse por turno a los tres trabajos que requieren servicio. Puesto que las personas que están depurando programas usualmente emiten comandos cortos (p. ej., compilar un procedimiento de cinco páginas) en vez de largos (p. ej., ordenar un archivo de un millón de registros), la computadora puede proporcionar servicio rápido interactivo a varios usuarios y tal vez también trabajar con trabajos de lote grandes en segundo plano cuando la CPU está ociosa. Aunque el primer sistema serio de tiempo compartido (CTSS) fue creado en el M.I.T. en una 7094 especialmente modificada (Corbato et al., 1962), el tiempo compartido no se popularizó realmente hasta que se generalizó el uso del hardware de protección necesario durante la tercera generación.

Después del éxito del sistema CTSS, MIT, Bell Labs y General Electric (por ese entonces un fabricante importante de computadoras) decidieron emprender el desarrollo de un “servicio de computadora” una máquina que diera apoyo a cientos de usuarios de tiempo compartido simultáneos. Su modelo fue el sistema de distribución de electricidad: cuando usted necesita potencia eléctrica, simplemente enchufa una clavija en la pared y, dentro de límites razonables, obtendrá tanta electricidad como necesite. Los diseñadores de este sistema, llamado **MULTICS** (*servicio de información y computación multiplexado*), contemplaban una enorme máquina que proporcionara potencia de cómputo a todos los usuarios de Boston. La idea de que máquinas mucho más potentes que su GE-645 se vendieran como computadoras personales por unos cuantos miles de dólares sólo 30 años después no era sino ciencia ficción en ese entonces.

Para resumir un poco la historia, MULTICS introdujo muchas ideas seminales en la literatura de computación, pero su construcción fue mucho más difícil de lo que nadie había imaginado. Bell Labs abandonó el proyecto, y General Electric dejó el negocio de las computadoras por completo. Finalmente, MULTICS funcionó lo bastante bien como para usarse en un entorno de producción de MIT y en docenas de otros sitios, pero el concepto de un servicio de computadora se hizo obsoleto al desplomarse los precios de las computadoras. No obstante, MULTICS tuvo una influencia enorme sobre los sistemas subsecuentes; se le describe en (Corbato et al., 1972; Corbato y Vyssotsky, 1965; Daley y Dennis, 1968; Organick, 1972; Saltzer, 1974).

Otro avance importante durante la tercera generación fue el crecimiento fenomenal de las minicomputadoras, comenzando con la DEC PDP-1 en 1961. La PDP-1 sólo tenía 4K de palabras de 18 bits, pero a \$120 000 por máquina (menos del 5% del precio de una 7094), se vendieron como pan caliente. Para ciertos tipos de trabajos no numéricos, la PDP-1 era casi tan rápida como la 7094, e hizo nacer una

industria totalmente nueva. A esta máquina pronto siguió una serie de otras PDP (todas incompatibles, a diferencia de la familia IBM), culminando en la PDP-11.

Uno de los computólogos de Bell Labs que había trabajado en el proyecto MULTICS, Ken Thompson, encontró subsecuentemente una pequeña minicomputadora PDP-7 que nadie estaba usando y se propuso escribir una versión de MULTICS reducida al mínimo, para un solo usuario. Este trabajo posteriormente evolucionó para convertirse en el sistema operativo **UNIX®**, que se popularizó en el mundo académico, las dependencias del gobierno y muchas compañías.

La historia de UNIX se cuenta en otras obras (p. ej., Salus, 1994). Baste con decir que, dado que casi todo mundo podía obtener el código fuente, diversas organizaciones desarrollaron sus propias versiones (incompatibles), lo que condujo al caos. Con objeto de que fuera posible escribir programas susceptibles de ejecución en cualquier sistema UNIX, el IEEE creó un estándar para UNIX, llamado **Posix**, que casi todas las versiones actuales de UNIX reconocen. POSIX define una interfaz mínima de llamadas al sistema que los sistemas UNIX deben reconocer. De hecho, algunos otros sistemas de programación ya reconocen la interfaz POSIX.

1.2.4 La cuarta generación (1980-presente): Computadoras personales

Con la invención de los circuitos integrados a gran escala (LSI), chips que contienen miles de transistores en un cm² de silicio, nació la era de la computadora personal. En términos de arquitectura, las computadoras personales no eran muy diferentes de las minicomputadoras de la clase PDP-11, pero en términos de precio sí que eran diferentes. Si bien la minicomputadora hacía posible que un departamento de una compañía o universidad tuviera su propia computadora, el chip microprocesador permitía que un solo individuo tuviera su propia computadora personal. Las computadoras personales más potentes empleadas por empresas, universidades e instalaciones del gobierno suelen llamarse **estaciones de trabajo**, pero en realidad sólo son computadoras personales grandes. Por lo regular estas máquinas están interconectadas mediante una red.

La amplia disponibilidad de la potencia de cómputo, sobre todo la potencia de cómputo altamente interactiva casi siempre acompañada por excelentes gráficos, dio pie al crecimiento de una importante industria productora de software para computadoras personales. Una buena parte de este software era **amistoso con el usuario**, lo que significa que estaba dirigido a usuarios que no sólo no sabían nada de computación, sino que además no tenían la mínima intención de aprender. Sin duda, esto representaba un cambio drástico respecto al OS/360, cuyo lenguaje de control de trabajos, JCL, era tan arcano que llegaron a escribirse libros enteros sobre él (p. ej., Cadow, 1970).

Dos sistemas operativos dominaron inicialmente el campo de las computadoras personales y las estaciones de trabajo: **MS-DOS** de Microsoft y **UNIX**. MS-DOS se usaba ampliamente en la IBM PC y otras máquinas basadas en la CPU Intel 8088 y sus sucesoras, la 80286, 80386 y 80486 (que en adelante llamaremos la 286, 386 y 486, respectivamente) y más tarde la Pentium y Pentium Pro. Aunque la versión inicial de MS-DOS era relativamente primitiva, versiones subsecuentes han incluido características más avanzadas, muchas de ellas tomadas de UNIX. El sucesor de Microsoft para MS-DOS, WINDOWS, originalmente se ejecutaba encima de MS-DOS (es decir, era más un shell que un verdadero sistema operativo), pero a partir de 1995 se produjo una versión autosuficiente de WINDOWS, **WINDOWS 95®**, de modo que ya no se necesita MS-DOS para apoyarlo. Otro sistema operativo de Microsoft es **WINDOWS NT**, que es compatible con WINDOWS 95 en cierto nivel, pero internamente se reescribió desde cero.

El otro competidor importante es UNIX, que domina en las estaciones de trabajo y otras computadoras del extremo alto, como los servidores de red. UNIX es popular sobre todo en máquinas basadas en chips RISC de alto rendimiento. Estas máquinas por lo regular tienen la potencia de cómputo de una minicomputadora, a pesar de estar dedicadas a un solo usuario, por lo que resulta lógico que estén equipadas con un sistema operativo diseñado originalmente para minicomputadoras, a saber, UNIX.

Una tendencia interesante que apareció a mediados de la década de 1980 fue el crecimiento de redes de computadoras personales en las que se ejecutan **sistemas operativos de red** o **sistemas operativos distribuidos** (Tanenbaum, 1995). En un sistema operativo de red los usuarios están conscientes de la existencia de múltiples computadoras y pueden ingresar en máquinas remotas y copiar archivos de una máquina a otra. Cada máquina ejecuta su propio sistema operativo local y tiene su propio usuario o usuarios locales.

Los **sistemas operativos de red** no son fundamentalmente distintos de aquellos para un solo procesador. Obviamente, estos sistemas necesitan un controlador de la interfaz con la red y software de bajo nivel para operarlo, así como programas para realizar inicios de sesión remotos y acceso a archivos remotos, pero estas adiciones no alteran la estructura esencial del sistema operativo.

Un **sistema operativo distribuido**, en cambio, presenta el mismo aspecto a los usuarios que un sistema tradicional de un solo procesador, aunque en realidad se compone de múltiples procesadores. Los usuarios no deben enterarse de en dónde se están ejecutando sus programas o almacenando sus archivos; de todo eso debe encargarse el sistema operativo automáticamente y eficientemente.

Los verdaderos sistemas operativos distribuidos requieren más que la adición de un poco más de código a un sistema operativo uniprocador, porque los sistemas distribuidos y centralizados difieren en aspectos cruciales. Los sistemas distribuidos, por ejemplo, a menudo permiten a las aplicaciones ejecutarse en varios procesadores al mismo tiempo, por lo que requieren algoritmos de planificación de más complejos a fin de optimizar el grado de paralelismo.

En muchos casos, los retardos de comunicación dentro de la red implican que éstos (y otros) algoritmos deban ejecutarse con información incompleta, caduca o incluso incorrecta. Esta situación difiere radicalmente un sistema de un solo procesador en el que el sistema operativo tiene toda la información sobre el estado del sistema.

1.2.5 Historia de MINIX

Cuando UNIX era joven (Versión 6), era fácil conseguir el código fuente, bajo licencia de AT&T, y se estudiaba mucho. John Lions, de la University of New South Wales en Australia, incluso escribió un librito que describía su operación, línea por línea (Lions, 1996). Este librito se usó (con permiso de AT&T) como texto en muchos cursos universitarios de sistemas operativos.

Cuando AT&T liberó la Versión 7, comenzó a darse cuenta de que UNIX era un producto comercial valioso, así que entregó la Versión 7 junto con una licencia que prohibía el estudio del código fuente en cursos, a fin de evitar poner en peligro su situación de secreto comercial. Muchas universidades simplemente abandonaron el estudio de UNIX e impartieron sólo teoría.

Desafortunadamente, cuando sólo se enseña teoría el estudiante adquiere una visión desbalanceada de cómo se ve realmente un sistema operativo. Los temas teóricos que suelen cubrirse con gran detalle en cursos y libros sobre sistemas operativos, como los algoritmos de planificación, en la práctica realmente no son tan importantes. Los temas que en verdad son relevantes, como E/S y sistemas de archivos, generalmente se descuidan porque no hay mucha teoría al respecto.

A fin de remediar esta situación, uno de los autores de este libro (Tanenbaum) decidió escribir un nuevo sistema operativo desde cero que fuera compatible con UNIX desde el punto de vista del usuario, pero completamente distinto en su interior. Al no utilizar ni una sola línea del código de AT&T, este sistema evita las restricciones de la licencia, así que puede usarse en clase o para estudio individual. De esta forma, los lectores pueden disectar un sistema operativo real para ver qué hay dentro, tal como los estudiantes de biología disectan ranas. El nombre **MINIX** significa **mini-UNIX** porque es lo suficientemente pequeño como para poderlo entender a pesar de no ser un gurú.

Además de la ventaja de eliminar los problemas legales, MINIX tiene otra ventaja respecto a UNIX: se escribió una década después de UNIX y tiene una estructura más modular. El sistema de archivos de MINIX, por ejemplo, no forma parte del sistema operativo, sino que se ejecuta como programa de usuario.

Otra diferencia es que UNIX se diseñó de modo que fuera eficiente; MINIX se diseñó pensando en que fuera comprensible (hasta donde puede ser comprensible cualquier programa que ocupa cientos de páginas). El código de MINIX, por ejemplo, incluye miles de comentarios.

MINIX se diseñó originalmente de modo que fuera compatible con UNIX Versión 7 (V7). Se usó como modelo esta versión a causa de su sencillez y elegancia. A veces se dice que la Versión 7 no sólo representó una mejora respecto a sus predecesores, sino también respecto a todos sus sucesores. Con la llegada de POSIX, MINIX comenzó a evolucionar hacia el nuevo estándar, al tiempo que mantenía la compatibilidad hacia atrás con los programas existentes. Este tipo de evolución es común en la industria de las computadoras, pues ningún proveedor desea introducir un sistema nuevo que ninguno de sus clientes existentes podrá usar sin grandes convulsiones. La versión de MINIX que se describe en este libro se basa en el estándar POSIX (a diferencia de la versión descrita en la primera edición, que se basaba en V7).

Al igual que UNIX, MINIX se escribió en el lenguaje de programación C y se pretendía que fuera fácil transportarlo a diversas computadoras. La implementación inicial fue para la IBM PC, porque esta computadora se usa ampliamente. Subsecuentemente se llevó a las computadoras Atari, Amiga, Macintosh y SPARC. Acorde con la filosofía de que *“lo pequeño es hermoso”*, MINIX originalmente no requería siquiera un disco duro para ejecutarse, lo que lo ponía al alcance del presupuesto de muchos estudiantes (aunque puede parecer asombroso ahora, a mediados de la década de 1980 cuando MINIX vio por primera vez la luz, los discos duros aún eran una novedad de precio elevado). Al crecer MINIX en funcionalidad y tamaño,

llegó al punto en que se hizo necesario un disco duro, pero en concordancia con la filosofía de MINIX basta con una partición de 30 megabytes. En contraste, algunos sistemas UNIX comerciales ahora recomiendan una partición de disco de 200 MB como mínimo indispensable.

Para el usuario medio sentado ante una IBM PC, ejecutar MINIX es similar a ejecutar UNIX. Muchos de los programas básicos, como cat, grep, is, make y el shell están presentes y desempeñan las mismas funciones que sus contrapartes de UNIX. Al igual que el sistema operativo mismo, todos estos programas de utilidad fueron reescritos completamente desde cero por el autor, sus estudiantes y algunas otras personas dedicadas.

En todo este libro se usará MINIX como ejemplo. No obstante, casi todo lo que se diga acerca de MINIX, a menos que se refiera al código en sí, también aplica a UNIX. Muchos de estos comentarios también aplican a otros sistemas. Esto debe tenerse siempre presente al leer el libro.

Como acotación, es posible que unas cuantas palabras acerca de LINUX y su relación con MINIX sean de interés para algunos lectores. Poco después de liberarse MINIX, se formó un grupo de noticias de USENET para hablar de él. En pocas semanas, este grupo tenía 40000 suscriptores, la mayor parte de los cuales quería agregar enormes cantidades de nuevas capacidades a MINIX a fin de hacerlo más grande y mejor (bueno, al menos más grande). Cada día, varios cientos de ellos ofrecían sugerencias, ideas y fragmentos de código. El autor de MINIX resistió con éxito esta arremetida durante varios años, a fin de mantener a MINIX lo suficientemente pequeño y aseado como para que los estudiantes lo entendieran. Gradualmente, la gente comenzó a convencerse de que su posición era inamovible. Finalmente, un estudiante finlandés, Linus Torvalds, decidió escribir un clon de MINIX que pretendía ser un sistema de producción con abundantes capacidades, más que una herramienta educativa. Así fue como nació **LINUX**.

1.3 CONCEPTOS DE SISTEMAS OPERATIVOS

La interfaz entre el sistema operativo y los programas de usuario está definida por el conjunto de “operaciones extendidas” que el sistema operativo ofrece. Estas instrucciones se han llamado tradicionalmente **llamadas al sistema**, aunque ahora pueden implementarse de varias formas. Para entender realmente lo que los sistemas operativos hacen, debemos examinar con detenimiento esta interfaz. Las llamadas disponibles en la interfaz varían de un sistema operativo a otro (aunque los conceptos subyacentes tienden a ser similares).

Por tanto, nos vemos obligados a escoger entre (1) generalidades vagas (“los sistemas operativos tienen llamadas al sistema para leer archivos”) y (2) algún sistema específico (“MINIX tiene una llamada al sistema READ) con tres parámetros: uno para especificar el archivo, uno para indicar dónde deben colocarse los datos y uno para indicar cuántos bytes deben leerse”)

Hemos escogido el segundo enfoque. Esto implica más trabajo, pero nos permite entender mejor qué es realmente lo que hacen los sistemas operativos. En la sección 1.4 examinaremos de cerca las llamadas al sistema presentes tanto en UNIX como en MINIX. Por sencillez, sólo nos referiremos a MINIX, pero las llamadas al sistema UNIX correspondientes se basan en POSIX en la mayor parte de los casos. Sin embargo, antes de estudiar las llamadas al sistema reales, vale la pena presentar un panorama general de MINIX, a fin de tener una idea global de qué es lo que hace un sistema operativo. Este panorama aplica igualmente bien a UNIX.

Las llamadas al sistema de MINIX pertenecen a dos categorías amplias: las que se ocupan de los procesos y las que se ocupan del sistema de archivos. A continuación las examinaremos por turno.

1.3.1 Procesos

Un concepto clave en MINIX, y en todos los sistemas operativos, es el **proceso**. Un proceso es básicamente un programa en ejecución. Cada proceso tiene asociado un **espacio de direcciones**, una lista de posiciones de memoria desde algún mínimo (usualmente 0) hasta algún máximo, que el proceso puede leer y escribir. El espacio de direcciones contiene el programa ejecutable, los datos del programa, y su pila. A cada proceso también se asocia un conjunto de registros, que incluyen el contador del programa, el apuntador de la pila y otros registros de hardware, así como toda la demás información necesaria para ejecutar el programa.

Volvemos al concepto de proceso con mucho mayor detalle en el capítulo 2, pero por ahora la forma más fácil de adquirir una idea intuitiva de lo que es un proceso es pensar en los sistemas de tiempo compartido. Periódicamente, el sistema operativo decide dejar de ejecutar un proceso y comenzar a ejecutar otro, por ejemplo, porque el primero ya tuvo más tiempo de CPU del que le tocaba durante el segundo anterior.

Cuando un proceso se suspende temporalmente de esta manera, debe reiniciarse después en el mismo

estado exactamente en que estaba en el momento en que se le detuvo. Esto implica que toda la información acerca del proceso se debe guardar explícitamente en algún lugar durante la suspensión. Por ejemplo, es posible que el proceso tenga varios archivos abiertos para lectura. Cada uno de estos archivos tiene asociado un apuntador que indica la posición actual (es decir, el número del byte o registro que se leerá a continuación). Cuando un proceso se suspende temporalmente, es necesario guardar todos estos apuntadores para que una llamada READ ejecutada después de reiniciarse el proceso lea los datos correctos. En muchos sistemas operativos, toda la información acerca de cada proceso, aparte del contenido de su propio espacio de direcciones, se almacena en una tabla del sistema operativo llamada **tabla de procesos**, que es un arreglo (o lista enlazada) de estructuras, una para cada proceso existente en ese momento.

Así, un proceso (suspendido) consiste en su espacio de direcciones, por lo regular llamado **imagen de núcleo** (recordando las memorias de núcleos magnéticos que se usaban en el pasado), y su entrada en la tabla de procesos, que contiene sus registros, entre otras cosas.

Las llamadas al sistema de administración para procesos clave son las que se ocupan de la creación y terminación de procesos. Consideremos un ejemplo representativo. Un proceso llamado **intérprete de comandos** o **shell** lee comandos de una terminal. El usuario acaba de teclear un comando solicitando la compilación de un programa. El shell debe crear ahora un proceso nuevo que ejecute el compilador. Cuando ese proceso haya terminado la compilación, ejecutará una llamada al sistema para terminarse a sí mismo.

Si un proceso puede crear uno o más procesos distintos (denominados **procesos hijos**) y éstos a su vez pueden crear procesos hijos, pronto llegamos a la estructura de árbol de procesos de la Fig. 1-5. Los procesos relacionados que están cooperando para realizar alguna tarea a menudo necesitan comunicarse entre sí y sincronizar sus actividades. Esta comunicación se llama **comunicación entre procesos**, y se estudiará con detalle en el capítulo 2.

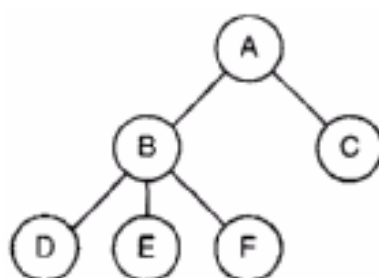


Figura 1-5. Un árbol de procesos.
El proceso A creó dos procesos hijos, B y C. El proceso B creó tres procesos hijos, D, E y F.

Hay otras llamadas al sistema relacionadas con procesos que solicitan más memoria (o liberan memoria no utilizada), esperan que un proceso hijo termine, y superponen otro programa al suyo.

Ocasionalmente, se hace necesario comunicar información a un proceso en ejecución que no está simplemente esperando recibirla. Por ejemplo, un proceso que se comunica con otro proceso en una computadora distinta lo hace enviando mensajes por una red. A fin de prevenir la posibilidad de que un mensaje o su respuesta se pierda, el remitente puede solicitar que su propio sistema operativo le notifique cuando haya transcurrido cierto número de segundos, a fin de poder retransmitir el mensaje si todavía no ha llegado un acuse de recibo. Después de establecer este temporizador, el programa puede seguir realizando otros trabajos.

Cuando ha transcurrido el número de segundos que se especificó, el sistema operativo envía una **señal** al proceso. La señal hace que el proceso suspenda temporalmente lo que estaba haciendo, guarde sus registros en la pila, y comience a ejecutar un procedimiento especial de manejo de señales, por ejemplo, para retransmitir un mensaje que al parecer se perdió. Una vez que el manejador de señales termina, el proceso en ejecución se reinicia en el estado en que estaba justo antes de la señal. Las señales son el análogo en software de las interrupciones de hardware, y pueden ser generadas por diversas causas además de la expiración de temporizadores. Muchas trampas detectadas por el hardware, como la ejecución de una instrucción no permitida o el empleo de una dirección no válida, también se convierten en señales que se envían al proceso culpable.

El administrador del sistema asigna un **uid** (identificador de usuario) a cada persona autorizada para usar MINIX. Cada proceso iniciado en MINIX tiene el uid de la persona que lo inició. Un proceso hijo tiene el mismo uid que su padre. Un uid, llamado **superusuario**, tiene facultades especiales, y puede violar muchas

de las reglas de protección. En las instalaciones grandes, sólo el administrador del sistema conoce la contraseña necesaria para convertirse en superusuario, pero muchos de los usuarios ordinarios (sobre todo estudiantes) dedican un esfuerzo considerable a tratar de encontrar defectos en el sistema que les permitan convertirse en superusuarios sin contar con la contraseña.

1.3.2 Archivos

La otra categoría amplia de llamadas al sistema se relaciona con el sistema de archivos. Como ya se apuntó, una función importante del sistema operativo es ocultar las peculiaridades de los discos y otros dispositivos de E/S y presentar al programador un modelo abstracto, aseado y bonito, de archivos independientes del dispositivo. Es obvio que se necesitan llamadas al sistema para crear, eliminar, leer y escribir archivos. Antes de que un archivo pueda leerse, debe abrirse, y después de leerse debe cerrarse, así que también se incluyen llamadas para hacer estas cosas.

A fin de contar con un lugar para guardar los archivos, MINIX tiene el concepto de **directorio** como mecanismo para agrupar los archivos. Un estudiante, por ejemplo, podría tener un directorio para cada curso en el que está inscrito (donde guardaría los programas necesarios para ese curso), otro directorio para su correo electrónico, y otro más para su página base de la World Wide Web. Por tanto, se necesitan llamadas al sistema para crear y eliminar directorios. También se incluyen llamadas para poner un archivo existente en un directorio, y para quitar un archivo de un directorio. Las entradas de directorio pueden ser archivos u otros directorios. Este modelo también da pie a una jerarquía -el sistema de archivos- como se muestra en la Fig. 1-6.

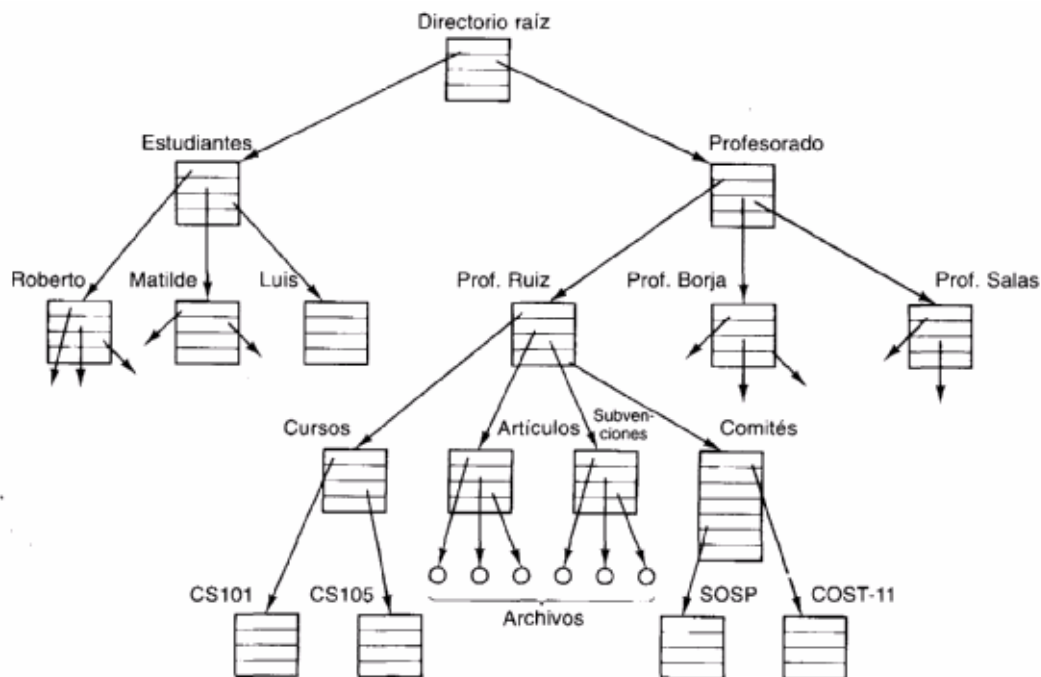


Figura 1-6. Sistema de archivos para un departamento universitario

Las jerarquías de procesos y de archivos están organizadas como árboles, pero hasta ahí llega la similitud. Las jerarquías de procesos no suelen ser muy profundas (casi nunca tienen más de tres niveles), en tanto que las de archivos comúnmente tienen cuatro, cinco o incluso más niveles de profundidad. Las jerarquías de procesos por lo regular tienen una vida corta, generalmente de unos cuantos minutos como máximo, en tanto que la jerarquía de directorios podría existir durante años. La propiedad y protección también es diferente para los procesos y para los archivos. Típicamente, sólo un proceso padre puede controlar o incluso acceder a un proceso hijo, pero casi siempre existen mecanismos para permitir que los archivos y directorios sean leídos por un grupo más amplio que sólo el propietario.

Cada archivo dentro de la jerarquía de directorios se puede especificar dando su **nombre de ruta** a partir del tope de la jerarquía de directorios, el **directorio raíz**. Semejantes nombres de ruta absolutos consisten en la lista de directorios por los que se debe pasar partiendo del directorio raíz para llegar al archivo, separando los componentes con diagonales. En la Fig. 1-6, la ruta del archivo CS101 es /Profesorado/Prof. Ruiz/Cursos/CS101. La diagonal (/) inicial indica que la ruta es absoluta, es decir, que comienza en el directorio raíz.

En todo momento, cada proceso tiene un **directorio de trabajo** actual, en el cual se buscan los archivos cuyos nombres de ruta no comienzan con una diagonal. Por ejemplo, en la Fig. 1-6, si /Profesorado/Prof. Ruiz fuera el directorio de trabajo, el empleo del nombre de ruta Cursos/ CS101 se referiría al mismo archivo que el nombre de ruta absoluta dado en el párrafo anterior. Los procesos pueden cambiar de directorio de trabajo emitiendo una llamada al sistema que especifique el nuevo directorio de trabajo.

Los archivos y directorios en MINIX se protegen asignando a cada uno un código de protección binario de 9 bits. El código de protección consiste en tres campos de 3 bits, uno para el propietario, uno para otros miembros del grupo del propietario (el administrador del sistema divide a los usuarios en grupos) y uno para toda la demás gente. Cada campo tiene un bit para acceso de lectura, uno para acceso de escritura y uno para acceso de ejecución. Estos tres bits se conocen como **bits rwx**. Por ejemplo, el código de protección rwxr-x--x significa que el propietario puede leer, escribir o ejecutar el archivo, otros miembros del grupo pueden leer o ejecutar (pero no escribir) el archivo, y el resto de la gente puede ejecutar (pero no leer ni escribir) el archivo. En el caso de un directorio, **x** indica permiso de búsqueda. Un guión significa que el permiso correspondiente está ausente.

Antes de poder leer o escribir un archivo, es preciso abrirlo, y en ese momento se verifican los permisos. Si está permitido el acceso, el sistema devuelve un entero pequeño llamado **descriptor de archivo** que se usará en operaciones subsecuentes. Si el acceso está prohibido, se devuelve un código de error.

Otro concepto importante en MINIX es el de **sistema de archivos montado**. Casi todas las computadoras personales tienen una o más unidades de disco flexible en las que pueden insertarse y de las que pueden retirarse disquetes. A fin de contar con una forma congruente de manejar estos medios removibles (y también los CD-ROM, que también son removibles), MINIX permite conectar el sistema de archivos del disco flexible al árbol principal. Considere la situación de la Fig. 1-7(a). Antes de la llamada MOUNT, el disco en RAM (disco simulado en la memoria principal) contiene el **sistema de archivos raíz**, o primario, y la unidad 0 contiene un disquete que contiene otro sistema de archivos.

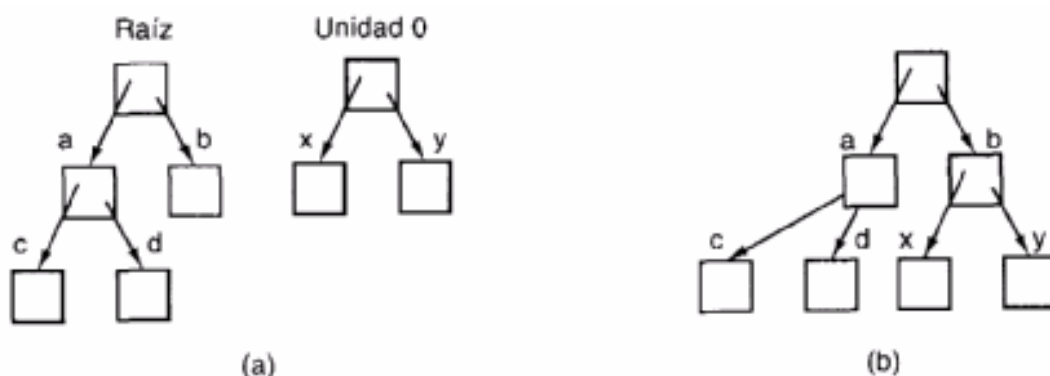


Figura 1-7. (a) Antes de montarse, los archivos de la unidad 0 no están accesibles. (b) Después de montarse, esos archivos forman parte de la jerarquía de archivos.

Sin embargo, no podemos usar el sistema de archivos de la unidad 0, porque no hay forma de especificar nombres de ruta en él. MINIX no permite anteponer a los nombres de ruta un nombre o número de unidad; ésa sería precisamente la clase de dependencia del dispositivo que los sistemas operativos deben eliminar. En vez de ello, la llamada al sistema MOUNT permite conectar el sistema de archivos de la unidad 0 al sistema de archivo raíz en cualquier lugar en el que el programa quiera que esté. En la Fig. 1-7(b) el sistema de archivos de la unidad 0 se montó en el directorio b, permitiendo así el acceso a los archivos /b/x y /b/y. Si el directorio b hubiera contenido archivos, éstos no habrían estado accesibles mientras estuviera montada la unidad 0, ya que /b se habría referido al directorio raíz de la unidad 0. (No poder acceder a esos archivos no es tan grave como parece a primera vista: los sistemas de archivos casi siempre se montan en directorios vacíos.)

Otro concepto importante en MINIX es el **archivo especial**. Los archivos especiales sirven para hacer que los dispositivos de E/S semejen archivos. Así, esos dispositivos pueden leerse y escribirse usando las mismas llamadas al sistema que se usan para leer y escribir archivos. Existen dos tipos de archivos especiales: **archivos especiales por bloques** y **archivos especiales por caracteres**. Los primeros se usan para modelar dispositivos que consisten en una colección de bloques directamente direccionables, como los discos. Al abrir un archivo especial por bloques y leer, digamos, el bloque 4, un programa puede acceder directamente al bloque 4 del dispositivo, pasando por alto la estructura del sistema de archivos que contiene. De forma similar, los archivos especiales por caracteres se usan para modelar impresoras, módems y otros dispositivos que aceptan o producen flujos de caracteres.

La última característica que mencionaremos en esta reseña general se relaciona tanto con los procesos como con los archivos: **los conductos**. El conducto (o **tubería**) es una especie de pseudoarchivo que puede servir para conectar dos procesos, como se muestra en la Fig. 1-8. Cuando el proceso A desea enviar datos al proceso B, escribe en el conducto como si fuera un archivo de salida. El proceso B puede leer los datos leyendo del conducto como si fuera un archivo de entrada. Así, la comunicación entre procesos en MINIX se parece mucho a las lecturas y escrituras de archivos normales. Es más, la única forma en que un proceso puede descubrir que el archivo de salida en el que está escribiendo no es realmente un archivo, sino un conducto, es emitiendo una llamada especial al sistema.



Figura 1-8. Dos procesos conectados por un conducto

1.3.3 El shell

El sistema operativo MINIX es el código que ejecuta las llamadas al sistema. Los editores, compiladores, ensambladores, vinculadores e intérpretes de comandos definitivamente no forman parte del sistema operativo, aunque son importantes y útiles. A riesgo de confundir un poco las cosas, en esta sección examinaremos brevemente el intérprete de comandos de MINIX, llamado **shell**, que, si bien no es parte del sistema operativo, utiliza intensivamente muchas de las características del sistema operativo y, por tanto, es un buen ejemplo de la forma en que pueden usarse las llamadas al sistema. El shell también es la interfaz primaria entre un usuario sentado ante su terminal y el sistema operativo.

Cuando un usuario ingresa en el sistema, se inicia un shell. El shell tiene la terminal como entrada estándar y salida estándar, y lo primero que hace es exhibir la **indicación (prompt)**, un carácter como un signo de dólar, que le indica al usuario que el shell está esperando para aceptar un comando. Si el usuario ahora tecldea

date

por ejemplo, el shell crea un proceso hijo y ejecuta el programa **date** como hijo. Mientras se está ejecutando el proceso hijo, el shell espera a que termine. Cuando el hijo termina, el shell exhibe otra vez la indicación y trata de leer la siguiente línea de entrada.

El usuario puede especificar que la salida estándar sea redirigida a un archivo, por ejemplo,

date >archivo

De forma similar, la entrada estándar puede redirigirse, como en

sort <archivo 1 >archivo2

que invoca el programa **sort** con entradas tomadas de archivo1 y enviando las salidas a archivo2.

La salida de un programa puede usarse como entrada para otro programa conectándolos con un conducto. Así,

cat archivo1 archivo2 archivo3 | sort >/dev/lp

invoca el programa **cat** para concatenar tres archivos y enviar la salida a **sort** para que acomode todas las líneas en orden alfabético. La salida de **sort** se dirige al archivo **/dev/lp**, que es un nombre típico para el archivo especial por caracteres de la impresora. (Por convención, todos los archivos especiales se guardan en el directorio **/dev**.)

Si un usuario escribe un signo **&** después de un comando, el shell no espera hasta que se completa, sino que exhibe una indicación de inmediato. Por tanto,

cat archivo1 archivo2 archivo3 | sort >/dev/lp &

inicia el ordenamiento como trabajo de segundo plano, permitiendo que el usuario siga trabajando normalmente mientras se está realizando el ordenamiento. El shell tiene varias otras características interesantes que no tenemos espacio para examinar aquí